

```

// 1. Define the Area of Interest (AOI): Thoothukudi District
var countries = ee.FeatureCollection("FAO/GAUL/2015/level2");
var aoi = countries.filter(ee.Filter.eq('ADM2_NAME', 'Thoothukudi'));
Map.centerObject(aoi, 10);

// 2. Function to Mask Clouds for Landsat 8 and 9
function maskL89sr(image) {
  var qa = image.select('QA_PIXEL');
  var mask = qa.bitwiseAnd(1 << 3).or(qa.bitwiseAnd(1 << 4));
  return image.updateMask(mask.not());
}

// 3. Loop through each year from 2016 to 2025
var startYear = 2016;
var endYear = 2025;

for (var year = startYear; year <= endYear; year++) {

  var dateStart = ee.Date.fromYMD(year, 1, 1);
  var dateEnd = ee.Date.fromYMD(year, 12, 31);

  // 4. Load Landsat 8 or 9 (L9 is available from 2022 onwards)
  var collection = ee.ImageCollection("LANDSAT/LC08/C02/T1_L2")
    .merge(ee.ImageCollection("LANDSAT/LC09/C02/T1_L2"))
    .filterBounds(aoi)
    .filterDate(dateStart, dateEnd)
    .filter(ee.Filter.lt('CLOUD_COVER', 30))
    .map(maskL89sr);

  // 5. Calculate Median LST for the year
  var lst_celsius = collection.median()
    .select('ST_B10')
    .multiply(0.00341802)
    .add(149.0)
    .subtract(273.15)
    .rename('LST_Celsius')
    .clip(aoi);

  // 6. Calculate Mean LST for the District (the reference value)
  var lst_mean = ee.Number(lst_celsius.reduceRegion({
    reducer: ee.Reducer.mean(),
    geometry: aoi,
    scale: 100, // Scale increased to 100 for faster processing of multiple years
    maxPixels: 1e9
  })).values().get(0);

  // 7. Calculate UHI (Anomaly: Pixel - Mean)
  var uhi = lst_celsius.subtract(lst_mean).rename('UHI_Intensity_' + year);

  // 8. Visualization (Optional: Displays only the last year on map to avoid clutter)
  var uhi_vis = {
    min: 0, max: 10,
    palette: ['#315695', '#74add1', '#fed976', '#feb24c', '#fdbd3c', '#fc4e2a', '#e31a1c', '#b10026']
  };

  if (year === endYear) {
    Map.addLayer(uhi, uhi_vis, 'UHI Intensity ' + year);
  }

  // 9. Export Each Year to Google Drive
  // After running the script, go to the "Tasks" tab and click "Run" for each year.
  Export.image.toDrive({
    image: uhi,
    description: 'Thoothukudi_UHI_' + year,
    folder: 'Thoothukudi_UHI_Project',
    scale: 30,
    region: aoi,
    fileFormat: 'GeoTIFF',
    maxPixels: 1e13
  });

  print('Prepared export task for year: ' + year);
}

```

```

// 1. Define the Area of Interest (AOI): Thoothukudi District
var countries = ee.FeatureCollection("FAO/GAUL/2015/level2");
var aoi = countries.filter(ee.Filter.eq('ADM2_NAME', 'Thoothukudi'));
Map.centerObject(aoi, 10);

// 2. Function to Mask Clouds for Landsat 8 and 9
function maskL89sr(image) {
  var qa = image.select('QA_PIXEL');
  var mask = qa.bitwiseAnd(1 << 3).or(qa.bitwiseAnd(1 << 4));
  return image.updateMask(mask.not());
}

// 3. Loop through each year from 2016 to 2025
var startYear = 2016;
var endYear = 2025;

for (var year = startYear; year <= endYear; year++) {

  var dateStart = ee.Date.fromYMD(year, 1, 1);
  var dateEnd = ee.Date.fromYMD(year, 12, 31);

  // 4. Load Landsat 8 and 9 Collection
  var collection = ee.ImageCollection("LANDSAT/LC08/C02/T1_L2")
    .merge(ee.ImageCollection("LANDSAT/LC09/C02/T1_L2"))
    .filterBounds(aoi)
    .filterDate(dateStart, dateEnd)
    .filter(ee.Filter.lt('CLOUD_COVER', 30))
    .map(maskL89sr);

  // 5. Calculate Median LST in Kelvin
  var lst_kelvin = collection.median()
    .select('ST_B10')
    .multiply(0.00341802)
    .add(149.0)
    .rename('LST_Kelvin')
    .clip(aoi);

  // 6. Calculate Mean LST for the District
  var lst_mean =
  ee.Number(lst_kelvin.reduceRegion({
    reducer: ee.Reducer.mean(),
    geometry: aoi,
    scale: 100,
    maxPixels: 1e9
  })).values().get(0));

  // 7. Calculate UTFVI
  // Formula: (LST - LST_mean) / LST
  var utfvi = lst_kelvin.subtract(lst_mean).divide(lst_kelvin).rename('UTFVI_' + year);

  // 8. Visualization (Last year shown on map)
  var utfvi_vis = {
    min: -0.05,
    max: 0.05,
    palette: ['313695', '74add1', 'fed976', 'feb24c', 'fd8d3c', 'fc4e2a', 'e31a1c', 'b10026']
  };

  if (year == endYear) {
    Map.addLayer(utfvi, utfvi_vis, 'UTFVI ' + year);
  }

  // 9. Export Each Year to Google Drive
  Export.image.toDrive({
    image: utfvi,
    description: 'Thoothukudi_UTFVI_' + year,
    folder: 'Thoothukudi_UTFVI_Project',
    scale: 30,
    region: aoi,
    fileFormat: 'GeoTIFF',
    maxPixels: 1e13
  });

  print('Task created for UTFVI year: ' + year);
}

```

```

// 1. Define AOI

var countries = ee.FeatureCollection("FAO/GAUL/
2015/level2");

var aoi = countries.filter(ee.Filter.eq('ADM2_NAME',
'Thoothukudi'));

Map.centerObject(aoi, 10);

function maskL89sr(image) {
  var qa = image.select('QA_PIXEL');
  var mask = qa.bitwiseAnd(1 <<
3).or(qa.bitwiseAnd(1 << 4));
  return image.updateMask(mask.not());
}

// 2. Loop for 10 years (2016 - 2025)
for (var year = 2016; year <= 2025; year++) {
  var collection = ee.ImageCollection("LANDSAT/
LC08/C02/T1_L2")
    .merge(ee.ImageCollection("LANDSAT/LC09/C02/
T1_L2"))
    .filterBounds(aoi)
    .filterDate(year + '-01-01', year + '-12-31')
    .map(maskL89sr);

  var lst = collection.median().select('ST_B10')
    .multiply(0.00341802).add(149.0).subtract(273.15).
clip(aoi);

  // Get Min/Max for the whole district
  var stats = lst.reduceRegion({
    reducer: ee.Reducer.percentile([2, 98]),
    geometry: aoi,
    scale: 100,
    maxPixels: 1e9
  });

  var l_min = ee.Number(stats.get('ST_B10_p2'));
  var l_max = ee.Number(stats.get('ST_B10_p98'));

  // CWSI without the vegetation mask
  var cws_i =
lst.subtract(l_min).divide(l_max.subtract(l_min))
    .clamp(0, 1).rename('CWSI_' + year);

  // Export
  Export.image.toDrive({
    image: cws_i,
    description: 'Thoothukudi_CWSI_' + year,
    folder: 'Thoothukudi_CWSI_Project',
    scale: 30,
    region: aoi,
    fileFormat: 'GeoTIFF'
  });
}

// Visualizing the last year
Map.addLayer(cws_i, {min: 0, max: 1, palette: ['blue',
'cyan', 'yellow', 'red']}, 'CWSI 2025 (Unmasked)');

```

```

// ☒ NDVI ANALYSIS - THOOTHUKUDI DISTRICT
(78.1348, 8.7642)

// MULTI-YEAR ANALYSIS (2016-2025)

// 1. Load District Boundaries

var adminRegions = ee.FeatureCollection("FAO/
GAUL/2015/level2");

// 2. Correct Thoothukudi Coordinates

var centralCoord = [78.1348, 8.7642];

var cityPoint = ee.Geometry.Point(centralCoord);

// 3. Get Thoothukudi District Boundary

var thoothukudi =
adminRegions.filterBounds(cityPoint)
.filter(ee.Filter.stringContains('ADM1_NAME', 'Tamil
Nadu'))
.map(function(f) { return f.simplify(1000); });

Map.centerObject(thoothukudi, 9);

Map.addLayer(thoothukudi, {color: 'red'},
'Thoothukudi District ROI', true);

// 4. Define Analysis Years

var years = ee.List.sequence(2016, 2025);

// 5. Function to Process Each Year

var processYear = function(year) {
  year = ee.Number(year);
  var startDate = ee.Date.fromYMD(year, 4, 1);
  var endDate = ee.Date.fromYMD(year, 9, 30);

  // Landsat 8 Surface Reflectance

  var landsat = ee.ImageCollection("LANDSAT/LC08/
C02/T1_L2")
  .filterBounds(thoothukudi)
  .filterDate(startDate, endDate)
  .filter(ee.Filter.lt('CLOUD_COVER', 20))
  .map(function(img) {

    // Cast to Double for 64-bit precision

    var imgDouble = img.toDouble();

    // Apply scale factors

    var sr =
imgDouble.multiply(0.0000275).add(-0.2);

    // Calculate NDVI with 64-bit precision

    var ndvi = sr.normalizedDifference(['SR_B5','SR_
B4']).rename('NDVI');

    return ndvi.copyProperties(img,
['system:time_start']);
  });

  var medianNDVI =
landsat.median().clip(thoothukudi);

  // Attach year metadata

  return medianNDVI.set('year',
year).set('landsatCount', landsat.size());
};

// 6. Apply Function Across All Years

var results =
ee.ImageCollection(years.map(processYear));

// 7. Visualize one example year (e.g., 2020)

var example = results.filter(ee.Filter.eq('year',
2020)).first();

var ndviVis = {min: 0, max: 1, palette:
['brown','yellow','green']};

Map.addLayer(example, ndviVis, 'NDVI 2020');

// 8. Export Each Year Separately

years.getInfo().forEach(function(y) {
  var img = results.filter(ee.Filter.eq('year', y)).first();

  // Note: .visualize() converts to 8-bit RGB (0-255).

  // To keep 64-bit data in the file, export 'img'
  directly without .visualize()

  Export.image.toDrive({
    image: img.clip(thoothukudi), // Exporting the
    raw double-precision NDVI band
    description: 'Thoothukudi_NDVI_' + y,
    folder: 'NDVI_Analysis',
    region: thoothukudi,
    scale: 30,
    maxPixels: 1e13
  });
});

```



```

// 1. Define the Boundary for Thoothukudi District,
Tamil Nadu

// We use the FAO GAUL dataset to filter for the
specific district

var districts = ee.FeatureCollection("FAO/GAUL/
2015/level2");

var thoothukudi =
districts.filter(ee.Filter.eq('ADM2_NAME',
'Thoothukudi'));

// Center the map on the district
Map.centerObject(thoothukudi, 9);

Map.addLayer(thoothukudi, {color: 'red'},
'Thoothukudi Boundary', false);

// 2. Load WorldPop Dataset (Annual Gridded
Population)

var worldPop = ee.ImageCollection("WorldPop/GP/
100m/pop");

// 3. Function to calculate Total Population for a
specific year
var getYearlyPop = function(year) {
  var popYear = worldPop
    .filter(ee.Filter.eq('year', year))
    .filter(ee.Filter.eq('country', 'IND')) // Filter for
India
    .first();

  var stats = popYear.reduceRegion({
    reducer: ee.Reducer.sum(),
    geometry: thoothukudi,
    scale: 100, // WorldPop resolution is ~100m
    maxPixels: 1e9
  });

  return ee.Feature(null, {
    'Year': year,
    'Population': stats.get('population')
  });
};

// 4. Create a list of years from 2016 to 2025
var years = ee.List.sequence(2016, 2025);

var yearlyStats =
ee.FeatureCollection(years.map(getYearlyPop));

// 5. Print Tabulation to Console
print('Population Tabulation for Thoothukudi
(2016-2025):', yearlyStats);

// 6. Export the data as a CSV to Google Drive
Export.table.toDrive({
  collection: yearlyStats,
  description: 'Thoothukudi_Population_2016_2025',
  fileFormat: 'CSV',
  selectors: ['Year', 'Population']
});

// 7. Visualization: Display the 2025 Population
Layer
var pop2025 = worldPop.filter(ee.Filter.eq('year',
2025)).first().clip(thoothukudi);

var visParams = {
  min: 0,
  max: 50,
  palette: ['white', 'yellow', 'orange', 'red']
};

Map.addLayer(pop2025, visParams, 'Population
Density 2025');

```

```

// 1. Define the Boundary for Thoothukudi
// Using the simplified LISIG boundary or GAUL

var countries = ee.FeatureCollection("FAO/GAUL/
2015/level2");

var thoothukudi =
countries.filter(ee.Filter.eq('ADM2_NAME',
'Thoothukudi'));

// Center the map
Map.centerObject(thoothukudi, 9);

// 2. Load WorldPop Dataset
// We use the 'unconstrained' dataset which is the
most comprehensive

var worldPop = ee.ImageCollection("WorldPop/GP/
100m/pop");

// 3. Function to calculate Total Population safely
var getYearlyPop = function(year) {

  // Filter for the year and the specific country code
  (ISO 356 is India)

  var popImg = worldPop
    .filter(ee.Filter.eq('year', year))
    .filter(ee.Filter.eq('country', 'IND'))
    .first();

  // Check if image exists to avoid the "null" error
  var totalPop = ee.Algorithms.If(popImg,
    popImg.reduceRegion({
      reducer: ee.Reducer.sum(),
      geometry: thoothukudi,
      scale: 100,
      maxPixels: 1e9
    }).get('population'),
    0 // Return 0 if data for that year isn't found
  );

  return ee.Feature(null, {
    'Year': year,
    'Population': totalPop
  });
};

// 4. Create the sequence (2016 to 2020/2025)
// Note: WorldPop typically has verified data up to
2020,
// and projected data for later years.
var years = ee.List.sequence(2016, 2025);

var yearlyStats =
ee.FeatureCollection(years.map(getYearlyPop));

// 5. Output the Tabulation
print("Population Tabulation (2016-2025):",
yearlyStats);

// 6. Visualizing the most recent available year
(2020 or 2025)

var recentPop = worldPop
  .filter(ee.Filter.eq('country', 'IND'))
  .sort('year', false)
  .first()
  .clip(thoothukudi);

var visParams = {
  min: 0,
  max: 50,
  palette: ['#f7fbff', '#deebf7', '#c6dbef', '#9ecae1',
'#6baed6', '#4292c6', '#2171b5', '#084594']
};

Map.addLayer(recentPop, visParams, 'Latest
Population Density');

// 7. Export to Drive
Export.table.toDrive({
  collection: yearlyStats,
  description: 'Thoothukudi_Pop_Data',
  fileFormat: 'CSV'
});

```

```

// 1. Define the Boundary for Thoothukudi
// Using the simplified LISIG boundary or GAUL

var countries = ee.FeatureCollection("FAO/GAUL/
2015/level2");

var thoothukudi =
countries.filter(ee.Filter.eq('ADM2_NAME',
'Thoothukudi'));

// Center the map
Map.centerObject(thoothukudi, 9);

// 2. Load WorldPop Dataset
// We use the 'unconstrained' dataset which is the
most comprehensive

var worldPop = ee.ImageCollection("WorldPop/GP/
100m/pop");

// 3. Function to calculate Total Population safely
var getYearlyPop = function(year) {

  // Filter for the year and the specific country code
  (ISO 356 is India)

  var popImg = worldPop
    .filter(ee.Filter.eq('year', year))
    .filter(ee.Filter.eq('country', 'IND'))
    .first();

  // Check if image exists to avoid the "null" error
  var totalPop = ee.Algorithms.If(popImg,
    popImg.reduceRegion({
      reducer: ee.Reducer.sum(),
      geometry: thoothukudi,
      scale: 100,
      maxPixels: 1e9
    }).get('population'),
    0 // Return 0 if data for that year isn't found
  );

  return ee.Feature(null, {
    'Year': year,
    'Population': totalPop
  });
};

// 4. Create the sequence (2016 to 2020/2025)
// Note: WorldPop typically has verified data up to
2020,
// and projected data for later years.
var years = ee.List.sequence(2016, 2025);

var yearlyStats =
ee.FeatureCollection(years.map(getYearlyPop));

// 5. Output the Tabulation
print("Population Tabulation (2016-2025):",
yearlyStats);

// 6. Visualizing the most recent available year
(2020 or 2025)

var recentPop = worldPop
  .filter(ee.Filter.eq('country', 'IND'))
  .sort('year', false)
  .first()
  .clip(thoothukudi);

var visParams = {
  min: 0,
  max: 50,
  palette: ['#f7fbff', '#deebf7', '#c6dbef', '#9ecae1',
'#6baed6', '#4292c6', '#2171b5', '#084594']
};

Map.addLayer(recentPop, visParams, 'Latest
Population Density');

// 7. Export to Drive
Export.table.toDrive({
  collection: yearlyStats,
  description: 'Thoothukudi_Pop_Data',
  fileFormat: 'CSV'
});

```

```

// 1. Boundary for Thoothukudi

var districts = ee.FeatureCollection("FAO/GAUL/
2015/level2");

var thoothukudi =
districts.filter(ee.Filter.eq('ADM2_NAME',
'Thoothukudi'));

Map.centerObject(thoothukudi, 9);


// 2. Load Datasets

var worldPop = ee.ImageCollection("WorldPop/GP/
100m/pop");

var ghsl = ee.ImageCollection("JRC/GHSL/P2023A/
GHS_POP"); // High-res population surfaces


var getYearlyPop = function(year) {

  var popImg;

  if (year <= 2020) {

    // Use WorldPop for 2016-2020

    popImg = worldPop

      .filter(ee.Filter.eq('year', year))

      .filter(ee.Filter.eq('country', 'IND'))

      .first();

  } else {

    // Use GHSL for 2021-2025 (Estimates/
    Projections)

    // GHSL is stored by epochs (e.g., 2020, 2025,
    2030)

    // We select the closest projection epoch for the
    2020s

    popImg = ee.Image("JRC/GHSL/P2023A/
    GHS_POP/2025");

  }

  var stats = popImg.reduceRegion({

    reducer: ee.Reducer.sum(),

    geometry: thoothukudi,

    scale: 100,

    maxPixels: 1e9

  });

  return ee.Feature(null, {

    'Year': year,

    'Population': stats.getNumber(year <= 2020 ?
    'population' : 'population_count')

  });

};


// 3. Generate Data for 2016 - 2025

var years = ee.List.sequence(2016, 2025);

var yearlyStats =
ee.FeatureCollection(years.map(getYearlyPop));


// 4. Output

print('Combined Population Tabulation
(2016-2025):', yearlyStats);


Export.table.toDrive({

  collection: yearlyStats,

  description:
'Thoothukudi_Complete_Pop_2016_2025',

  fileFormat: 'CSV'

});

```



```

// 1. Boundary for Thoothukudi

var districts = ee.FeatureCollection("FAO/GAUL/
2015/level2");

var thoothukudi =
districts.filter(ee.Filter.eq('ADM2_NAME',
'Thoothukudi'));

Map.centerObject(thoothukudi, 9);


// 2. Load Datasets

var worldPop = ee.ImageCollection("WorldPop/GP/
100m/pop");

var ghsl = ee.ImageCollection("JRC/GHSL/P2023A/
GHS_POP"); // High-res population surfaces


var getYearlyPop = function(year) {

  var popImg;

  if (year <= 2020) {

    // Use WorldPop for 2016-2020

    popImg = worldPop

      .filter(ee.Filter.eq('year', year))

      .filter(ee.Filter.eq('country', 'IND'))

      .first();

  } else {

    // Use GHSL for 2021-2025 (Estimates/
    Projections)

    // GHSL is stored by epochs (e.g., 2020, 2025,
    2030)

    // We select the closest projection epoch for the
    2020s

    popImg = ee.Image("JRC/GHSL/P2023A/
    GHS_POP/2025");

  }

  var stats = popImg.reduceRegion({

    reducer: ee.Reducer.sum(),

    geometry: thoothukudi,

    scale: 100,

    maxPixels: 1e9

  });

  return ee.Feature(null, {

    'Year': year,

    'Population': stats.getNumber(year <= 2020 ?
    'population' : 'population_count')

  });

};


// 3. Generate Data for 2016 - 2025

var years = ee.List.sequence(2016, 2025);

var yearlyStats =
ee.FeatureCollection(years.map(getYearlyPop));


// 4. Output

print('Combined Population Tabulation
(2016-2025):', yearlyStats);


Export.table.toDrive({

  collection: yearlyStats,

  description:
'Thoothukudi_Complete_Pop_2016_2025',

  fileFormat: 'CSV'

});

```

**Sr. No Year Male Population Female Population Kids
Population (<18) Total Population**

1 2016 891,420 913,580 505,400 1,805,000

2 2017 897,840 920,160 509,040 1,818,000

3 2018 904,760 927,240 512,960 1,832,000

4 2019 911,200 933,800 516,600 1,845,000

5 2020 918,120 940,880 520,520 1,859,000

6 2021 943,210 966,790 534,800 1,910,000

7 2022 954,570 978,430 541,240 1,933,000

8 2023 961,980 986,020 545,440 1,948,000

9 2024 967,410 991,590 548,520 1,959,000

10 2025 972,350 996,650 551,320 1,969,000


```
/*
 * HHRI Time-Series Analysis: Thoothukudi District
 * (2016 - 2025)
 *
 * Weightage Update: Hazard (0.45), Exposure
 * (0.33), Vulnerability (0.22)
 */

// 1. Boundary Setup

var districts = ee.FeatureCollection("FAO/GAUL/
2015/level2");

var thoothukudi = districts.filter(ee.Filter.and(
  ee.Filter.eq(ADM1_NAME, 'Tamil Nadu'),
  ee.Filter.eq(ADM2_NAME, 'Thoothukudi')
));

// Geometry and Center
// Geometry
Map.centerObject(thoothukudi, 10);

// 2. Main HHRI Function

var runYearlyAnalysis = function(year) {
  var start = ee.Date.fromYMD(year, 3, 1);
  var end = ee.Date.fromYMD(year, 5, 31);

  // Landsat 8 Data Retrieval & LST Calculation
  var l8 = ee.ImageCollection("LANDSAT/LC08/C02/
T1_L2");

  // Filter by Bounds, Date, and Cloud Cover
  .filterBounds(thoothukudi)
  .filterDate(start, end)
  .filter(ee.Filter.lt('CLOUD_COVER', 30))
  .median();

  // Thermal constant conversion to Celsius
  var lst = l8.select('ST_B10').multiply(0.00341802).a
dd(149.0).subtract(273.15);

  // NDVI calculation for Vulnerability
  var ndvi = l8.normalizedDifference(['SR_B5',
'SR_B4']);

  // Population Density for Exposure
  var pop = ee.ImageCollection("CIESIN/GPW411/
GPW_Population_Density")
  .first().clip(thoothukudi);

  // Normalization Helper (Scaling to 0-1)
  var normalize = function(img) {
    var minMax = img.reduceRegion({
      reducer: ee.Reducer.minMax(),
      geometry: thoothukudi,
      scale: 1000,
      maxPixels: 1e9
    });

    var band = img.bandNames().get(0);
    var min = ee.Number(minMax.get(ee.String(band).cat('_min')));
    var max = ee.Number(minMax.get(ee.String(band).
cat('_max')));

    return img.subtract(min).divi
de(max.subtract(min)).clamp(0,
1);
  };

  // APPLYING UPDATED WEIGHTS
  // Hazard (LST) = 45%
  var hazard = normalize(lst).multiply(0.45);

  // Exposure (Population) = 33%
  var exposure = normalize(pop).multiply(0.33);

  // Vulnerability (Inverse NDVI) = 22%
  // Note: -1 * NDVI because lower vegetation
  // increases vulnerability
  var vulnerability =
normalize(ndvi.multiply(-1)).multiply(0.22);

  // Combine components into single HHRI Layer
  var hhri = hazard.add(exposure).add(vulnerability).
rename('HHRI').clip(thoothukudi);

  return hhri;
};

// 3. Execution Loop (2016 to 2025)

var startYear = 2016;
var endYear = 2025;

for (var year = startYear; year <= endYear; year++) {
  var result = runYearlyAnalysis(year);

  // Statistics
  var stats = result.reduceRegion({
    reducer: ee.Reducer.mean(),
    geometry: thoothukudi,
    scale: 100,
    maxPixels: 1e9
  });

  print('Year ' + year + ' Mean HHRI (New Weights):',
stats);

  // Visuals
  var palette = ['#0000FF', '#00FFFF', 'FFFFFF00',
'FF7F00', 'FFD000'];
  Map.addLayer(result, {min: 0, max: 0.8, palette:
palette}, 'HHRI ' + year);

  // Export to Drive
  Export.image.toDrive({
    image: result,
    description: 'HHRI_Weighted_Thoothukudi_' +
year,
    scale: 50,
    region: thoothukudi,
    fileFormat: 'GeoTIFF',
    maxPixels: 1e9
  });
}
```

```

// 1. Define the Study Area (Replace with your
actual Thoothukudi Asset)

var aol = ee.FeatureCollection("projects/google/
charts_feature_example") // Example; use your
district asset

.filter(ee.Filter.eq('name', 'Thoothukudi'));

Map.centerObject(aol, 10);

// 2. Load Landsat 8 Surface Temperature Data
var collection = ee.ImageCollection("LANDSAT/
LC08/C02/T1_L2")

.filterBounds(aol)

.filterDate('2023-01-01', '2023-12-31')

.filter(ee.Filter.lt('CLOUD_COVER', 10));

// 3. Pre-processing Function (Scale factors for L8
C2 L2)

function preprocess(image) {

  var thermal = image.select('ST_B10').multiply(0.0
0341802).add(149.0).subtract(273.15); // Kelvin to
Celsius

  var ndvi = image.normalizedDifference(['SR_B5',
'SR_B4']).rename('NDVI');

  return image.addBands(thermal.rename('LST')).add
Bands(ndvi);
}

var processedCol = collection.map(preprocess);
var medianImage = processedCol.median().clip(aol);

// 4. Calculate UTFVI

// UTFVI = (LST - Mean_LST) / Mean_LST

var stats =
medianImage.select('LST').reduceRegion({
  reducer: ee.Reducer.mean(),
  geometry: aol,
  scale: 30,
  maxPixels: 1e9
});

var meanLST = ee.Number(stats.get('LST'));
var utfvi = medianImage.select('LST').subtract(mean
LST).divide(meanLST).rename('UTFVI');

// 5. Simple HHRI (Example: Combining Heat and
Vegetation stress)

// HHRI = Normalized (LST + (1 - NDVI))

var lstNorm =
medianImage.select('LST').unitScale(20, 50); //
Normalize LST between 20-50°C

var ndviInverted = ee.Image(1).subtract(m
edianImage.select('NDVI')).unitScale(0,
1);

var hhri =
lstNorm.add(ndviInverted).divide(2).rename('HHRI');

// 6. Visualization

var tempVis = {min: 25, max: 45, palette: ['blue',
'yellow', 'red']};

var indexVis = {min: 0, max: 0.5, palette: ['white',
'orange', 'darkred']};

Map.addLayer(medianImage.select('LST'), tempVis,
'Land Surface Temperature');
Map.addLayer(utfvi, indexVis, 'UTFVI (Urban
Thermal Variance)');
Map.addLayer(hhri, indexVis, 'HHRI (Heat Risk
Index)');

// 7. Exporting for Charting (CSV)

// Create a time-series chart for your indices
var chart = ui.Chart.image.series({
  imageCollection: processedCol.select(['LST',
'NDVI']),
  region: aol,
  reducer: ee.Reducer.mean(),
  scale: 30
}).setOptions({
  title: 'Temporal LST and NDVI Trend',
  hAxis: {title: 'Date'},
  vAxis: {title: 'Value'},
});

print(chart);

```



```

// 1. Define the Study Area - Ensure this is your
actual Thoothukudi boundary

// If using a manual polygon, rename it to 'aol'
var aol = ee.FeatureCollection("FAO/GAUL/2015/
level2")

.filter(ee.Filter.eq('ADM2_NAME',
'Thoothukudi'));

Map.centerObject(aol, 10);

// 2. Load Landsat 8 Surface Temperature Data
// Increased cloud cover to 30% to ensure we get
data

var collection = ee.ImageCollection("LANDSAT/
LC08/C02/T1_L2")
.filterBounds(aol)
.filterDate("2023-01-01", "2023-12-31")
.filter(ee.Filter.lt("CLOUD_COVER", 30));

// DEBUG: Check if the collection actually has
images
print("Number of images found:", collection.size());

// 3. Pre-processing Function
function preprocess(image) {
  // Scaling factors for Landsat 8 Collection 2 Level
  2

  var thermal = image.select('ST_B10').multiply(0.003
41802).add(149.0).subtract(273.15);

  var ndvi = image.normalizedDifference(['SR_B5',
'SR_B4']).rename('NDVI');

  // Add bands and keep the timestamp (important
for the chart)

  return image.addBands(thermal.rename('LST'))
    .addBands(ndvi)
    .copyProperties(image,
[System.time_start]);
}

// Map the function
var processedCol = collection.map(preprocess);

// Use a conditional to prevent the "no bands" error
if the collection is empty
var medianImage = ee.Algorithms.If(
collection.size().gt(0),
processedCol.median().clip(aol),
ee.Image().rename('no_data') // Placeholder if
empty
);

// Convert back to image type for subsequent
processing
var finalImg = ee.Image(medianImage);

// 4. Calculate UTFVI (Only if images exist)
var stats = finalImg.select('LST').reduceRegion({
  reducer: ee.Reducer.mean(),
  geometry: aol,
  scale: 100, // Increased scale slightly for faster
computation
  maxPixels: 1e9
});

var meanLST = ee.Number(stats.get('LST'));

// Deriving UTFVI and HHRI
var utfvi = finalImg.select('LST').subtract(meanLST).
divide(meanLST).rename('UTFVI');
var lstNorm = finalImg.select('LST').unitScale(25,
45);
var ndviInverted = ee.Image(1).subtra
ct(finalImg.select('NDVI')).unitScale(0,
1);
var hhri =
lstNorm.add(ndviInverted).divide(2).rename('HHRI');

// 5. Visualization
if (collection.size().gt(0)) {
  Map.addLayer(finalImg.select('LST'), {min: 28, max:
40, palette: ['blue', 'yellow', 'red']}, 'LST');
  Map.addLayer(utfvi, {min: -0.1, max: 0.1, palette:
['green', 'white', 'red']}, 'UTFVI');
  Map.addLayer(hhri, {min: 0, max: 0.8, palette:
['white', 'orange', 'red']}, 'HHRI');
}

// 6. Charting (This solves the "non-null values of
system:time_start" error)
var chart = ui.Chart.image.series({
  imageCollection: processedCol.select(['LST',
'NDVI']),
  region: aol,
  reducer: ee.Reducer.mean(),
  scale: 500
}).setOptions({title: 'Seasonal Trend for
Thoothukudi'});

print(chart);

```

```

from qgis.core import QgsProject, QgsRasterLayer
import matplotlib.pyplot as plt
import numpy as np

# --- Collect NDVI layers ---
project = QgsProject.instance()
all_layers = project.mapLayers().values()

ndvi_data = {}

for layer in all_layers:
    if isinstance(layer, QgsRasterLayer) and "NDVI" in layer.name():
        name = layer.name()

        # Extract year from layer name (e.g., "Thoothukudi_NDVI_2021")
        try:
            year = int(name.split("_")[-1].strip())
        except ValueError:
            continue

        # Get raster statistics for Band 1
        provider = layer.dataProvider()
        stats = provider.bandStatistics(1)
        mean_ndvi = stats.mean

        ndvi_data[year] = mean_ndvi
        print(f"Year: Mean NDVI = {mean_ndvi:4f}")

# --- Sort by year ---
years = sorted(ndvi_data.keys())
means = [ndvi_data[y] for y in years]

# --- Plot Bar Graph ---
fig, ax = plt.subplots(figsize=(12, 6))

colors = plt.cm.RdYlGn(
    [(v - min(means)) / (max(means) - min(means)) for
     v in means]
)

bars = ax.bar(
    [str(y) for y in years],
    means,
    color=colors,
    edgecolor='black',
    linewidth=0.8,
    width=0.6
)

# Add value labels on top of bars
for bar, val in zip(bars, means):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.002,
        f'{val:.3f}',
        ha='center',
        va='bottom',
        fontsize=9,
        fontweight='bold'
    )

# --- Styling ---
ax.set_title(
    "Thoothukudi District - Mean NDVI (2016-2025)\nLandsat 8 | QGIS Analysis",
    fontsize=14,
    fontweight='bold',
    pad=15
)

ax.set_xlabel("Year", fontsize=12)
ax.set_ylabel("Mean NDVI Value", fontsize=12)
ax.set_ylim(min(means) - 0.05, max(means) + 0.05)
ax.axhline(
    y=np.mean(means),
    color='blue',
    linestyle='--',
    linewidth=1.2,
    label=f"Overall Mean: {np.mean(means):.3f}"
)

ax.legend(fontsize=10)
ax.grid(axis='y', linestyle='--', alpha=0.5)
ax.tick_params(axis='x', labelsize=10)

plt.tight_layout()
plt.savefig("C:/Users/YourName/Desktop/Thoothukudi_NDVI_BarGraph.png", dpi=150)
plt.show()

print(f"✅ Bar graph generated successfully!")

```

```

// 1. Define Area of Interest (Thoothukudi)

var districts = ee.FeatureCollection("FAO/GAUL/
2015/level2");

var roi = districts.filter(ee.Filter.eq('ADM2_NAME',
'Thoothukudi'));

Map.centerObject(roi, 9);


// 2. DATA ACQUISITION: Sentinel-2 Tiles

// This shows the 'Granules' you are acquiring

var s2_collection =
ee.ImageCollection("COPERNICUS/
S2_SR_HARMONIZED")

.filterBounds(roi)

.filterDate('2024-01-01', '2025-12-31');


// Create a footprint of the tiles

var s2_tiles = s2_collection.map(function(img) {

return ee.Feature(img.geometry());

});


// 3. DATA ACQUISITION: Landsat 8/9 Paths/Rows

var l9_collection = ee.ImageCollection("LANDSAT/
LC09/C02/T1_L2")

.filterBounds(roi)

.filterDate('2024-01-01', '2025-12-31');


var l9_footprint = l9_collection.map(function(img) {

return ee.Feature(img.geometry());

});


// --- VISUALIZATION ---


// Display Sentinel-2 Tiles (Granules)

Map.addLayer(s2_tiles.draw({color: 'blue',
strokeWidth: 2}), {}, 'Sentinel-2 Tile Footprints');


// Display Landsat Footprints (Paths/Rows)

Map.addLayer(l9_footprint.draw({color: 'green',
strokeWidth: 2}), {}, 'Landsat 9 Footprints');


// Display Study Area Boundary

var outline = ee.Image().paint({featureCollection:
roi, color: 1, width: 3});

Map.addLayer(outline, {palette: 'red'}, 'Thoothukudi
Boundary');


// 4. Print Metadata to Console for your Thesis
Tables

print('Sentinel-2 Acquisition Count:',
s2_collection.size());

print('Landsat 9 Acquisition Count:',
l9_collection.size());

print('Sample Image Metadata (Spectral Bands):',
s2_collection.first());

```



```

// 1. Define the Administrative Boundary for
Thoothukudi

var districts = ee.FeatureCollection("FAO/GAUL/
2015/level2");

var thoothukudi =
districts.filter(ee.Filter.eq('ADM2_NAME',
'Thoothukudi'));

// Center the map on the district
Map.centerObject(thoothukudi, 10);

// 2. Load Landsat 8 Surface Reflectance Tier 1 Data
var l8 = ee.ImageCollection("LANDSAT/LC08/C02/
T1_L2")

.filterBounds(thoothukudi)

.filterDate('2023-01-01', '2023-12-31') // Adjust
dates as needed

.filter(ee.Filter.lt('CLOUD_COVER', 10)) // Keep only
clear images

.median() // Create a composite to remove any
remaining artifacts

.clip(thoothukudi); // Clip exactly to the district
boundary

// 3. Visualization Parameters for Natural Color
(RGB)

// Landsat 8 Natural Color: Red=B4, Green=B3,
Blue=B2

var visParams = {

  bands: ['SR_B4', 'SR_B3', 'SR_B2'],

  min: 0.0,

  max: 0.3, // Surface reflectance scale

  gamma: 1.4

};

// 4. Display the Layers

// Add the clipped satellite image

Map.addLayer(l8, visParams, 'Thoothukudi Landsat
8 Satellite View');

// Add the boundary outline for clear visualization
var outline = ee.Image().paint({

  featureCollection: thoothukudi,

  color: 1,

  width: 2

});

Map.addLayer(outline, {palette: 'FF0000'}, 'District
Boundary');

// Set the base map to 'SATELLITE' for comparison if
needed

Map.setOptions('SATELLITE');

```

```
// 1. (Make sure your 'l8' image and 'thoothukudi'
variable are defined as before)
```

```
// 2. Select the specific bands you want to
download.
```

```
// For Urban Heat Island and general analysis, it is
best to download all relevant bands.
```

```
var exportImage = l8.select(['SR_B2', 'SR_B3',
'SR_B4', 'SR_B5', 'SR_B6', 'SR_B7']);
```

```
// 3. Export the image to Google Drive
```

```
Export.image.toDrive({
  image: exportImage,
  description: 'Thoothukudi_Landsat8_Export',
  folder: 'GEE_Thesis_Data', // The folder created in
your Google Drive
  fileNamePrefix: 'Thoothukudi_L8_2023',
  region: thoothukudi.geometry().bounds(), //
Exports the bounding box area
  scale: 30, // Landsat 8 resolution is 30 meters
  maxPixels: 1e13,
  fileFormat: 'GeoTIFF'
});
```

```
print('Export task started. Please go to the "Tasks"
tab on the right and click "Run".');
```